

tv_vae Camera-Conditioned TV VAE 模型 架构与超参数手册

生成日期：2026-06-02

适用目录： tv_vae

核心文件： model.py , train_vae.py , data.py , losses.py , visual_debug.py

1. 一句话概览

TVVAE 是一个面向自动驾驶多相机连续视频的 camera-conditioned scene-token VAE。它不使用 cylinder、BEV 或 triplane，而是把相机参数编码为 ray/camera embedding，将可变数量的物理相机观测聚合到固定数量的 scene tokens，再在时间维度和 scene-token 维度上做压缩。

输入是：

```
images = [B, T, V, 3, H, W], range [-1, 1]
K       = [B, T, V, 3, 3]      optional
E       = [B, T, V, 4, 4]      optional, camera_to_ego
```

输出是：

```
recon = [B, T, V, 3, H, W]
```

核心思想：

```
multi-camera video
-> per-camera causal encoder
-> camera/ray embedding from K/E
-> fixed scene-token aggregation
-> joint or non-joint T/S compression
-> Gaussian latent
-> target-camera-conditioned scene rendering
-> reconstructed multi-view video
```

论文主线不是研究新的几何投影，而是研究：

时域 T 与 scene-token 轴 S 是否应该联合压缩；
联合压缩是否能在相同 token 数下获得更好的 VAE 重建与 DiT 生成质量。

2. 符号约定

符号	含义
B	batch size
T	输入视频帧数，即 <code>sequence_length</code>
V	输入物理相机数，来自数据集样本
S	固定 scene token 数，即 <code>scene_token_count</code>
T_lat	latent time 数，即 <code>latent_time_count</code>
S_lat	latent scene token 数，即 <code>latent_scene_token_count</code>
H,W	输入图像分辨率
H_l,W_l	VAE bottleneck 空间分辨率
C0	<code>base_channels</code>
C_b	bottleneck channels
C_lat	<code>latent_channels</code>
K	相机内参
E	camera-to-ego 外参

常用 nuScenes debug 配置：

```
B = 1
T = 5
V = 6
H,W = 256,448
spatial_downsample_factor = 8
H_l,W_l = 32,56
S = 6
T_lat = 2
S_lat = 3
C0 = 48
C_b = 192
C_lat = 32
```

对应 posterior:

```
mean/logvar = [B, T_lat=2, S_lat=3, C_lat=32, H_l=32, W_l=56]
```

latent token 数:

```
N_lat = T_lat * S_lat * H_l * W_l
        = 2 * 3 * 32 * 56
        = 10,752
```

原始 camera-space token 数:

```
N_raw = T * V * H * W
        = 5 * 6 * 256 * 448
        = 3,440,640
```

token 位置压缩比:

```
N_raw / N_lat = 320x
```

如果把 RGB 和 latent channel 也计入标量数:

```
raw scalars    = T * V * 3 * H * W
latent scalars = T_lat * S_lat * C_lat * H_l * W_l
compression    = 10,321,920 / 344,064  $\approx$  30x
```

3. 总体数据流

以 $T=5$, $V=6$, $H=256$, $W=448$, $\text{spatial_downsample_factor}=8$, $S=6$, $T_{\text{lat}}=2$, $S_{\text{lat}}=3$ 为例：

input images

[B,5,6,3,256,448]

flatten camera views

[B*6,3,5,256,448]

stem causal Conv3d

[B*6,48,5,256,448]

down_0 stride_hw=2

[B*6,96,5,128,224]

down_1 stride_hw=2

[B*6,192,5,64,112]

down_2 stride_hw=2

[B*6,192,5,32,56]

unflatten camera views

[B,5,6,192,32,56]

camera geometry embedding from K/E

[B,5,6,192,32,56]

image feature + geometry embedding

[B,5,6,192,32,56]

scene aggregation: V cameras -> S scene tokens

[B,5,6,192,32,56]

-> [B,5,6,192,32,56]

TV compression

joint or non_joint:

[B,5,6,192,32,56]

-> [B,2,3,192,32,56]

to_moments

192 -> 2*C_lat = 64

posterior mean/logvar

[B,2,3,32,32,56]

z = mode or sample

[B,2,3,32,32,56]

from_latent

32 -> 192

[B,2,3,192,32,56]

TV expand

[B,2,3,192,32,56]

-> [B,5,6,192,32,56]

target camera query from K/E

[B,5,6,192,32,56]

camera renderer: target camera tokens attend to scene tokens

[B,5,6,192,32,56]

flatten camera views

[B*6,192,5,32,56]

up_0

[B*6,192,5,64,112]

up_1

[B*6,96,5,128,224]

up_2

[B*6,48,5,256,448]

head causal Conv3d

[B*6,3,5,256,448]

unflatten camera views

[B,5,6,3,256,448]

注意：如果 spatial_downsample_factor=4，只有两级空间下采样，H₁,W₁=64,112；如果设为 16，则有四级下采样，H₁,W₁=16,28。

4. Camera Geometry Embedding

4.1 输入相机参数

数据 adapter 输出：

```
camera_intrinsics = [B,T,V,3,3]
camera_transforms = [B,T,V,4,4] # camera_to_ego
intrinsics_hw      = (H_calib, W_calib)
```

模型不做 cylinder 投影，也不把图像投到 BEV。相机参数只用于构造每个低分辨率 feature cell 的几何 embedding。

4.2 每个低分辨率 cell 的几何特征

对于 bottleneck 网格上的每个位置 (h,w)，构造：

```
uv_norm:          2 dims, normalized image coordinates in [-1,1]
ray_dir_ego:       3 dims, camera ray direction transformed to ego frame
camera_center_ego: 3 dims, camera origin in ego frame

raw_geometry = [uv_norm, ray_dir_ego, camera_center_ego] # 8 dims
```

再经过 MLP：

```
Linear(8 -> C_b) -> SiLU -> Linear(C_b -> C_b)
```

输出：

```
geometry_embedding = [B,T,V,C_b,H_l,W_l]
```

如果数据集没有 K/E，ray_dir_ego 和 camera_center_ego 为零，模型退化为只使用

uv_norm + learned camera fallback embedding。因此 tv_vae 可以先在没有相机参数的数据集上跑通，但多数数据集泛化建议提供 K/E。

5. Scene Token Aggregation

5.1 为什么不用固定 camera-axis latent

如果 latent 直接使用 V 作为主轴，就会绑定数据集 camera rig：

```
nuScenes: V=6
Waymo:    V=5
AV2:      V=7
```

这会让 tokenizer 难以多数据集混训。当前实现把物理相机视为 observations，把 latent 主轴改为固定 scene token：

```
camera observations  $V$  -> fixed scene tokens  $S$ 
```

5.2 聚合方式

输入：

```
obs = image_feature + geometry_embedding
obs: [B,T,V,C_b,H_l,W_l]
```

对每个 (B,T,H_l,W_l) 位置，执行：

```
query = learnable scene tokens, shape  $[S,C_b]$ 
key/value =  $V$  camera observations
cross-attention:  $S$  queries attend to  $V$  cameras
```

输出：

```
scene = [B,T,S,C_b,H_l,W_l]
```

这个 S 是固定 scene memory token 数，不要求等于输入相机数。对于当前 nuScenes 单数据集调试，推荐：

```
scene_token_count = 6
```

如果后续多数据集混训，可以测试：

```
scene_token_count = 8 or 12
```

6. TV Compression

当前只保留两个模式：

```
--tv-compression joint  
--tv-compression non_joint
```

二者输入输出 token 数一致，方便做同 token 消融。

6.1 Joint Compression

输入：

```
scene = [B,T,S,C_b,H_l,W_l]
```

每个空间位置 (h,w) 独立做 cross-attention：

```
key/value tokens = T * S  
query tokens      = T_lat * S_lat
```

例如：

```
T=5, S=6 -> 30 KV tokens  
T_lat=2, S_lat=3 -> 6 query tokens
```

输出：

```
latent_feature = [B,T_lat,S_lat,C_b,H_l,W_l]
```

这一步不会把 H/W 全局 flatten 到 attention 里。实现上会临时 reshape 成：

```
[B*H_l*W_l, T*S, C_b]
```

这只是把空间位置并入 batch，每个空间柱只看自己的 T/S tokens，不做全局空间 attention。

6.2 Non-Joint Compression

输入同样是：

```
scene = [B,T,S,C_b,H_l,W_l]
```

但是不用 T/S cross-attention，而是做 separable pooling：

```
adaptive pooling on (T,S): [T,S] -> [T_lat,S_lat]  
MLP refine per token
```

输出：

```
latent_feature = [B,T_lat,S_lat,C_b,H_l,W_l]
```

因此：

joint 和 non_joint 的 token budget 完全相同
差别只在是否联合建模 T 与 S 的冗余

这对应论文中的“同 token，谁质量更好？”实验。

7. Gaussian Posterior

TV compression 后：

```
latent_feature = [B,T_lat,S_lat,C_b,H_l,W_l]
```

通道映射：

```
to_moments: C_b -> 2 * C_lat
```

得到：

```
mean    = [B,T_lat,S_lat,C_lat,H_l,W_l]
logvar  = [B,T_lat,S_lat,C_lat,H_l,W_l]
```

训练默认使用 deterministic path:

```
z = posterior.mode()
```

如果传:

```
--sample-posterior
```

则使用:

```
z = mean + eps * exp(0.5 * logvar)
```

8. Decoder

Decoder 输入:

```
z = [B,T_lat,S_lat,C_lat,H_l,W_l]
```

先恢复 bottleneck channels:

```
from_latent: C_lat -> C_b
[B,T_lat,S_lat,C_b,H_l,W_l]
```

8.1 TV Expand

joint 模式:

```
query = [T*S] scene queries
key/value = [T_lat*S_lat] latent tokens
cross-attention expand
```

non_joint 模式:

bilinear interpolation on (T,S): [T_lat,S_lat] -> [T,S]
MLP refine

两者输出:

scene = [B,T,S,C_b,H_1,W_1]

8.2 Target Camera Rendering

根据目标相机参数生成 camera query:

target K/E -> target geometry embedding
camera_query = [B,T,V_target,C_b,H_1,W_1]

对每个 (B,T,H_1,W_1) 位置:

query = V_target camera tokens
key/value = S scene tokens

输出:

camera_feature = [B,T,V_target,C_b,H_1,W_1]

这一步使 decoder 不再固定只能输出训练数据集的 camera-axis; 理论上可以根据 target camera config 输出对应视角。

8.3 Image Decoder

将 camera features 展平到 view batch:

[B,T,V,C_b,H_1,W_1]
-> [B*V,C_b,T,H_1,W_1]

然后逐级上采样:

up blocks + causal Conv3d
head Conv3d

输出：

```
[B,T,V,3,H,W]
```

9. Loss

当前训练只做 camera-domain loss，不再有 cylinder-domain loss：

```
loss =  
    reconstruction_loss  
    + perceptual_weight * LPIPS  
    + edge_loss_weight * edge_loss  
    + kl_weight_now * KL  
    + logvar_reg_weight * logvar_reg
```

9.1 Reconstruction Loss

由 `--rec-kind` 控制：

```
l1  
l2  
huber
```

默认建议：

```
--rec-kind l1
```

9.2 LPIPS 感知损失

```
--perceptual-weight 0.03  
--perceptual-batch-size 2
```

LPIPS 只作用在 camera-domain 图像上：

```
[B,T,V,3,H,W] -> [B*T*V,3,H,W]
```

9.3 Edge Loss

一阶图像梯度损失：

$$\text{edge} = 0.5 * (\text{L1}(\text{dx_pred}, \text{dx_gt}) + \text{L1}(\text{dy_pred}, \text{dy_gt}))$$

推荐：

```
--edge-loss-weight 0.1
```

9.4 KL 与 logvar regularization

KL 是标准 Gaussian posterior 到 $N(\theta, I)$ 的解析 KL。推荐 overfit/debug 阶段：

```
--kl-reduction mean
--kl-weight 1e-6
--kl-warmup 1500
--logvar-reg-weight 1e-4
```

10. 关键超参数

参数	推荐起点	含义
base_channels	48	encoder/decoder 基础通道
latent_channels	32	Gaussian latent 通道
scene_token_count	6	聚合后的 scene token 数
latent_time_count	2	压缩后的时间 token 数
latent_scene_token_count	3	压缩后的 scene token 数
spatial_downsample_factor	8	空间下采样倍率
num_attention_heads	4	attention heads
tv_compression	joint / non_joint	是否 T/S 联合压缩

10.1 空间压缩

spatial_downsample_factor 必须是 2 的幂：

1, 2, 4, 8, 16

对于 H=256,W=448 ：

spatial factor	H_1,W_1	latent tokens when T_lat=2,S_lat=3
4	64,112	43,008
8	32,56	10,752
16	16,28	2,688

8 是较强压缩；如果单样本 overfit 糊得明显，建议先用 4 验证模型上限。

11. 实验设计

当前两个模式：

joint
non_joint

它们共享：

image encoder
geometry embedding
scene aggregation
latent token count
Gaussian posterior
decoder
loss

唯一差异：

joint : T/S cross-attention compression
non_joint : separable T/S pooling compression

主要实验：

同 token，谁质量更好？

评价：

VAE reconstruction: L1, LPIPS, edge, PSNR/SSIM if needed

DiT generation: FVD/FID/downstream metric

training efficiency: step time, VRAM, steps-to-quality

如果要做“同质量，谁 token 更少？”，可扫描：

latent_time_count

latent_scene_token_count

spatial_downsample_factor

12. 训练命令

12.1 单样本 overfit: joint

```
CUDA_VISIBLE_DEVICES=0 PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True \  
python -m tv_vae.train_vae \  
  --data nusscenes_scene \  
  --nusc-data-root /shareNFS_40/sharedata/nusscenes/nusscenes_trainval \  
  --nusc-split train \  
  --sequence-length 5 \  
  --image-hw 256 448 \  
  --batch-size 1 \  
  --num-workers 0 \  
  --overfit-samples 1 \  
  --overfit-sample-index 0 \  
  --base-channels 48 \  
  --latent-channels 32 \  
  --scene-token-count 6 \  
  --latent-time-count 2 \  
  --latent-scene-token-count 3 \  
  --spatial-downsample-factor 8 \  
  --num-attention-heads 4 \  
  --tv-compression joint \  
  --rec-kind l1 \  
  --kl-reduction mean \  
  --kl-weight 1e-6 \  
  --kl-warmup 1500 \  
  --perceptual-weight 0.03 \  
  --perceptual-batch-size 2 \  
  --edge-loss-weight 0.1 \  
  --logvar-reg-weight 1e-4 \  
  --steps 1500 \  
  --lr 5e-4 \  
  --warmup-steps 100 \  
  --log-every 10 \  
  --preview-every 100 \  
  --ckpt-every 500 \  
  --amp bf16 \  
  --out tv_vae/runs/nus_overfit_joint_s6_lats3_s8
```

12.2 单样本 overfit: non_joint

只改：

```
--tv-compression non_joint  
--out tv_vae/runs/nus_overfit_non_joint_s6_lats3_s8
```

12.3 弱瓶颈 sanity

如果强压缩版本糊，先放松瓶颈：

```
--latent-time-count 5  
--latent-scene-token-count 6  
--spatial-downsample-factor 4
```

这能验证 encoder/decoder 和 scene-token 路径是否有足够重建能力。

13. Visual Debug

命令：

```
CUDA_VISIBLE_DEVICES=0 python -m tv_vae.visual_debug \  
  --data nusenes_scene \  
  --nusc-data-root /shareNFS_40/sharedata/nuscenes/nuscenes_trainval \  
  --nusc-split train \  
  --sample-index 0 \  
  --sequence-length 5 \  
  --image-hw 256 448 \  
  --base-channels 48 \  
  --latent-channels 32 \  
  --scene-token-count 6 \  
  --latent-time-count 2 \  
  --latent-scene-token-count 3 \  
  --spatial-downsample-factor 8 \  
  --num-attention-heads 4 \  
  --tv-compression joint \  
  --checkpoint tv_vae/runs/nus_overfit_joint_s6_lats3_s8/ckpts/last.pt \  
  --out tv_vae/runs/visual_debug_joint_s6_lats3_s8 \  
  --device cuda
```

主要输出：

```
input/input_grid.png
input/temporal_delta.png
reconstruction/recon_grid.png
reconstruction/abs_error.png
geometry/ray_ego_rgb.png
geometry/camera_center_norm.png
tokens/scene_token_energy_TxS.png
tokens/latent_token_energy_TxS.png
tokens/latent_spatial_energy.png
activations/*_spatial_energy.png
debug_metrics.json
report.md
```

判断方法：

现象	可能问题
input_grid.png 异常	dataset / adapter / 归一化问题
ray_ego_rgb.png 全黑或各 view 一样	K/E 未传入或 shape/坐标错误
scene_token_energy_TxS.png 整列接近 0	scene token collapse
latent_token_energy_TxS.png 整行/整列接近 0	TV compression token collapse
latent_spatial_energy.png 只亮局部	空间 latent 使用不均匀
recon_grid.png 糊但 token/activation 健康	bottleneck 太强或 decoder 容量不足
abs_error.png 某个相机明显更亮	该相机数据、顺序或 K/E 可疑

14. 当前版本的研究定位

旧 cylinder 版本的问题是几何 projector、无效区、near/far 半径等因素会干扰主 claim。当前 tv_vae 将问题收窄为：

```
给定多相机视频和相机参数，
先得到固定 scene-token 表示，
再研究 T 与 scene-token 轴是否应联合压缩。
```

因此核心消融更清楚：

joint vs non_joint
same latent token budget
same encoder/decoder
same loss

如果 joint 在同 token 下重建更好，或者接 DiT 后生成质量更好，就能说明：

自动驾驶多相机视频中的时间冗余和跨视角/**scene-token** 冗余适合联合建模：
TV joint compression 可以提高 latent token 的信息密度。